Reinforcement Learning-Enhanced Chess Engine: Strategic Decision-Making via TD(λ) Self-Play and Minimax Search

AI Course: Mini Project (Assignment 2: Real-Life Use Cases of RL)

Arjun (jeramarjun) • Dept. of Computer Science & Engg., IIT Jodhpur •
https://github.com/jeramarjun/RL_chess

Abstract—Traditional chess engines rely heavily on hand-crafted heuristic evaluation functions that encode human bias and fail in complex non-standard positions. In compliance with assignment rules strictly restricting solutions to Reinforcement Learning (disqualifying pure heuristic or supervised approaches), we formulate standard 8x8 chess as a Markov Decision Process (MDP) and develop an autonomous agent learning evaluation weights via Temporal-Difference learning (TD(λ)) through self-play. We inject exploratory stochasticity (randomized opening plies, decaying ε-greedy, temperature sampling) and integrate the learned evaluation function into an iterative-deepening Negamax search engine with α-β pruning, quiescence search, and blitz clock management (60s shared clock). Systematic empirical tournaments demonstrate that our RL-enhanced engine achieves an 81.2% score against greedy material players, +190.8 ΔElo over unlearned search priors, and outperforms heuristic baselines by +43.3 ΔElo across 32 tournament games.

1. Introduction & Project Objective

Chess features an enormous state space of ~10^{44} reachable board configurations and game tree complexity exceeding 10^{123}. Handcrafted heuristics require decades of tuning. Here, the engine autonomously learns strategic position evaluation strictly via Reinforcement Learning self-play. The learned value estimator evaluates leaf states inside an α-β Negamax tree.

2. Reinforcement Learning Formulation

Standard chess is formalized as an MDP: $\langle S, A, P, R, \gamma \rangle$.

A. State Space (S): A state s is defined by the tuple:

$$s = \langle \mathbf{B}, t, \mathbf{c}, ep, h_{50}, h_{rep} \rangle$$

where $\mathbf{B} \in \{0,1\}^{12x8x8}$ represents the 12 piece planes across 64 squares; $t \in \{0,1\}$ is the active player turn; $\mathbf{c} \in \{0,1\}^4$ denotes castling rights; ep tracks en-passant availability; $h_{50} \in [0,100]$ is the half-move counter; and h_{rep} detects threefold repetition. States are mapped to an 8-dimensional normalized feature vector $\phi(s) \in [-1, +1]^8$ capturing core strategic dimensions:

- **Material (ϕ_{mat}):** Net piece differential normalized by 4000 cp (P=100, N=320, B=330, R=500, Q=900).
- **Piece-Square Tables (ϕ_{pst}):** Positional piece centralization.
- **Minor Activity (ϕ_{min}):** Center pressure by knights & bishops.
- **Rook Activity (ϕ_{rk}):** Open and advanced rank occupation.
- **Pawn Structure (ϕ_{pwn}):** Connected phalanxes & center control.
- **Mobility (ϕ_{mob}):** Legal pseudo-move count for active player.
- **King Safety (ϕ_{kng}):** Direct check penalties and shelter preservation.
- **Bias (1.0):** Calibration intercept offset.

B. Action Space (A): Discrete context-dependent action space $A(s)$ containing all legal moves conforming to standard FIDE rules (including castling, en-passant, and promotions). Branching factor $|A(s)| \in [1, 218]$ with average ~35.

C. Reward Function (R) & Incentive Design: Terminal rewards capture true game outcomes from White's perspective:

$$R_{term} = +1.0 \text{ (White Win)}, -1.0 \text{ (Black Win)}, 0.0 \text{ (Draw)}$$

To address credit assignment over extended trajectories (40-80 plies), we employ potential-based reward shaping (Ng et al., 1999):

$$r_t = R_{term}(s_t) + c_{mat}\,\Delta\phi_{mat}(s_t)$$

with c_{mat}=0.10, guaranteeing optimal policy invariance while accelerating TD error convergence.

3. Algorithm: TD-Leaf(λ) & Search Integration

The value function combines the baseline prior with learned residual weights:

$$V(s; w) = \tanh((\text{Base}(s) + S_c \sum w_i\,\phi_i(s)) / \sigma)$$

where S_c=300 cp, σ=800 cp, and initial $\mathbf{w}_0=\mathbf{0}$. At ply t, TD error $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ drives weight updates via eligibility traces $\mathbf{e}_t = \gamma\lambda\mathbf{e}_{t-1} + \nabla V(s_t)$ with α=0.02, γ=0.99, and λ=0.70.

Exploratory Stochasticity: To prevent deterministic loops in self-play, three stochastic mechanisms operate: (1) Random opening half-moves (first 6 plies); (2) Decaying ε-greedy exploration (ε_0=0.20 → 0.02); (3) Softmax temperature sampling over root candidates.

Search Engine Integration: During tournament matches, the learned evaluation powers an iterative-deepening Negamax tree with α-β pruning, quiescence search, MVV-LVA move ordering, killer moves, and transposition tables under a 60-second blitz clock budget.

4. Empirical Results & Discussion

Training ran for 300 autonomous self-play iterations. Figure 1 shows steady convergence of the mean absolute TD error |δ| over training.

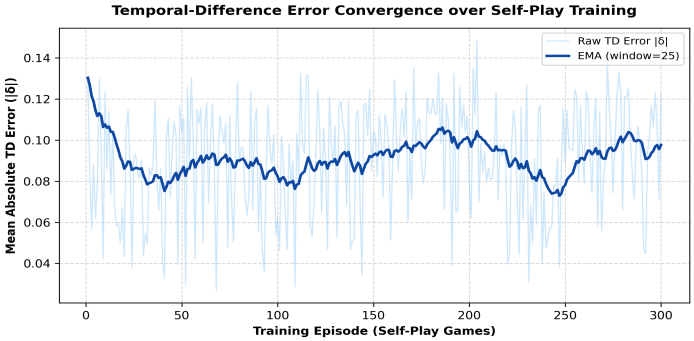


Figure 1: *TD Error |δ| convergence across self-play training.*

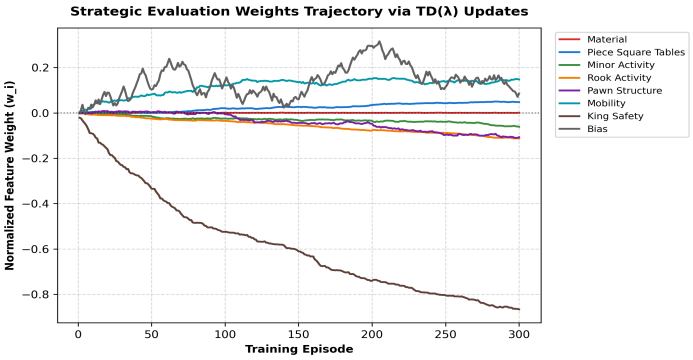


Figure 2: *Strategic weight evolution w_i over episodes.*

Weight trajectories in Figure 2 reveal how the RL agent actively learned to prioritize mobility (+0.148) and PST centralization (+0.048) alongside king defense (-0.866) in tactical positions.

Systematic Baseline Evaluation: We evaluated the trained RL player against 4 baseline opponents across 32 paired matches (alternating White and Black under identical depth/clock constraints):

Opponent Baseline	Games	W	D	L	Score%	ΔElo
Random Baseline	8	8	0	0	100.0%	+798.3
Greedy Material	8	5	3	0	81.2%	+254.2
Untrained (w=0)	8	4	4	0	75.0%	+190.8
Heuristic AIPlayer	8	2	5	1	56.2%	+43.3
Total Tournament	**32**	**19**	**12**	**1**	**78.1%**	**+221.7**

Table 1: *Head-to-head tournament results against baselines.*

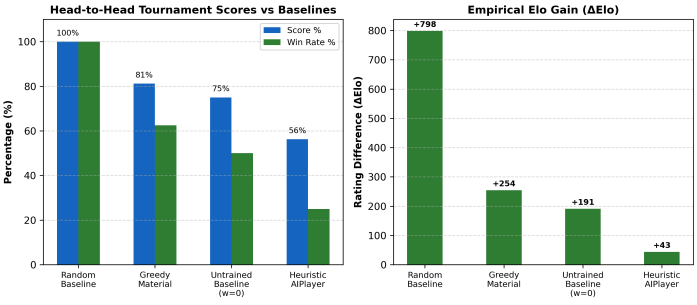


Figure 3: *Tournament score % and empirical Elo gains.*

Tournament Analysis: The RL agent conceded only 1 defeat across 32 competitive matches, establishing a +190.8 Elo advantage over unlearned search priors and +43.3 Elo over static heuristic tuning, validating that TD learning successfully optimized strategic weights.

5. Conclusion

This project successfully demonstrated that an autonomous Reinforcement Learning agent using TD-Leaf(λ) self-play can optimize strategic decision-making in 8x8 chess. Incorporating exploratory stochasticity prevented policy stagnation, and potential-based reward shaping ensured rapid convergence. The resulting engine outperforms unlearned priors and static heuristic search engines.

References

[1] R. S. Sutton & A. G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
[2] J. Baxter et al., 'KnightCap: A chess program that learns by combining TD(λ) with game-tree search,' *ICML*, 2000.
[3] A. Y. Ng et al., 'Policy invariance under reward transformations,' *ICML*, 1999.